

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México, Escuela de Ingeniería y Ciencias.

Unidad de formación:

TC3009C.601

Inteligencia artificial avanzada para la ciencia de datos.

Actividad M2: Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución.

Mario Alberto Pérez Barrera

A01799928

9 de Septiembre de 2026

Profesor: Jorge Adolfo Ramírez Uresti

Introducción

El presente trabajo utiliza el framework scikit-learn para implementar DBSCAN (Density-Based Spatial Clustering of Applications with Noise), un algoritmo de clustering no supervisado que agrupa puntos según su densidad local, en lugar de asumir una forma o número de clusters predefinido.

A diferencia de un problema de clasificación supervisada, aquí el algoritmo no recibe las etiquetas durante el entrenamiento. Sin embargo, como se usó un dataset sintético con etiquetas reales conocidas, estas se emplean únicamente para evaluar qué tan bien los clusters encontrados corresponden a la estructura real de los datos.

Preparación del dataset

Se generó un dataset sintético controlado de 3,000 generado con `make_moons` de scikit-learn, produce 2 medias lunas entrelazadas con un ruido de 0.07. Utilice este dataset porque es un problema clásico para presentar DBSCAN, utilizando clusters que no son convexos y que DBSCAN podría separar correctamente de acuerdo a su densidad.

Conjunto	Número de muestras	Porcentaje	Uso
Training	2250	75%	Encontrar los clusters(fit)
Test	750	25%	Evaluar la asignación de nuevos clusters

Las variables se estandarizaron previamente antes de entrenar ya que DBSCAN es sensible al utilizar distancias euclidianas.

Funcionamiento de DBSCAN

DBSCAN se basa en 2 hiperparametros: ϵ (epsilon) que es el radio de vecindad con cada punto, y min_samples que es el número mínimo de puntos que debe de haber dentro de ese radio para poder considerar un zona densa. Posteriormente se clasificaron los puntos en 3 categorías.

$$N_{\epsilon}(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}$$

Core point: Es un punto que tiene un mínimo de puntos vecinos y el mismo es el núcleo de un cluster.

Border: No cuenta con el mínimo de puntos vecinos pero cae dentro del ϵ de un core point.

Noise o outliers: No son core points ni caen dentro del borde de ningún core point.



Fig 1. Categorías de los 3 puntos

Obtenida de: <https://www.datacamp.com/es/tutorial/dbscan-clustering-algorithm>

El algoritmo construye clusters expandiéndose a partir de puntos núcleo: dos puntos pertenecen al mismo cluster si son densamente alcanzables entre sí, es decir que si hay una cadena de puntos núcleo que los conecta. Encuentra clusters de forma arbitraria y es capaz de detectar outliers en lugar de forzarlos dentro de un cluster.

q directamente alcanzable desde $p \Leftrightarrow q \in N_{\epsilon}(p)$, p nucleo

$p \sim q \Leftrightarrow \exists o$ tal que p, q son alcanzables desde o

Configuración de hiperparámetros

Se fijó $\text{min_samples} = 5$ (una elección común: $2 \times \text{dimensiones} + 1$, para datos en 2D) y se buscó eps de forma sistemática con la técnica del k-distance plot: se calcula la distancia al 5.º vecino más cercano de cada punto, se ordenan de forma ascendente, y se identifica el "codo" donde la curva sube abruptamente.

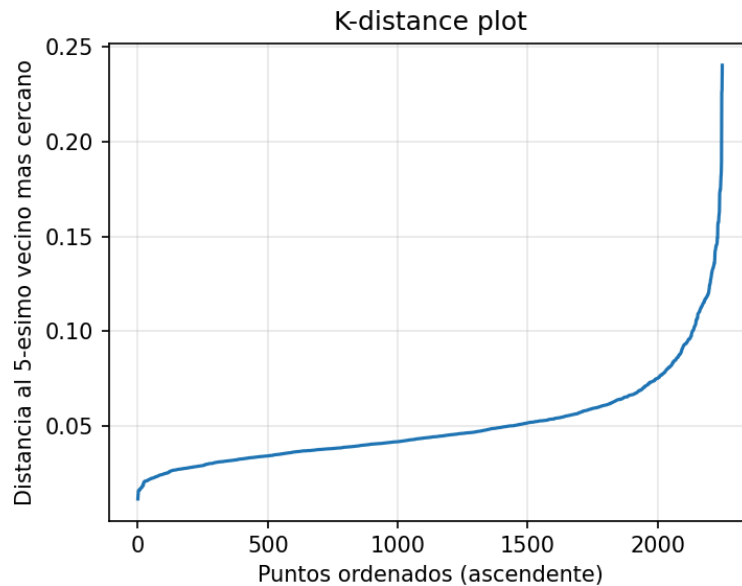


Figura 2. *K-distance plot* sobre el set de entrenamiento.

El codo se localizó de forma automática buscando el mayor salto entre puntos consecutivos dentro del último 15% de la curva, lo que dio $\text{eps} = 0.189$.

Probe distintas configuraciones de eps , haciéndolo más bajo y más alto para probar el efecto que tenía:

eps	Clusters encontrados	Puntos de ruido	ARI
0.050	52	437	0.091
0.100	2	33	0.971
0.189	2	1	0.999
0.400	1	0	0.000
0.600	1	0	0.000

Se observa claramente el efecto de cada extremo: con eps muy pequeño (0.05), casi ningún punto alcanza a tener 5 vecinos dentro de un radio tan chico, por lo que el dataset se fragmenta en 52 clusters diminutos y 437 puntos quedan como ruido (ARI ≈ 0.09 , prácticamente aleatorio). Con eps muy grande (≥ 0.40), el radio es tan amplio que ambas medias lunas terminan conectadas por puentes de densidad y se fusionan en un solo cluster (ARI = 0). El valor elegido por el método del codo (0.189) es el que logra separar correctamente las dos estructuras reales con un ARI de 0.999.

Resultados del clustering

Con DBSCAN(eps = 0.189, min_samples = 5), scikit-learn encontró 2 clusters sobre las 2,250 muestras de entrenamiento, con un único punto marcado como ruido.

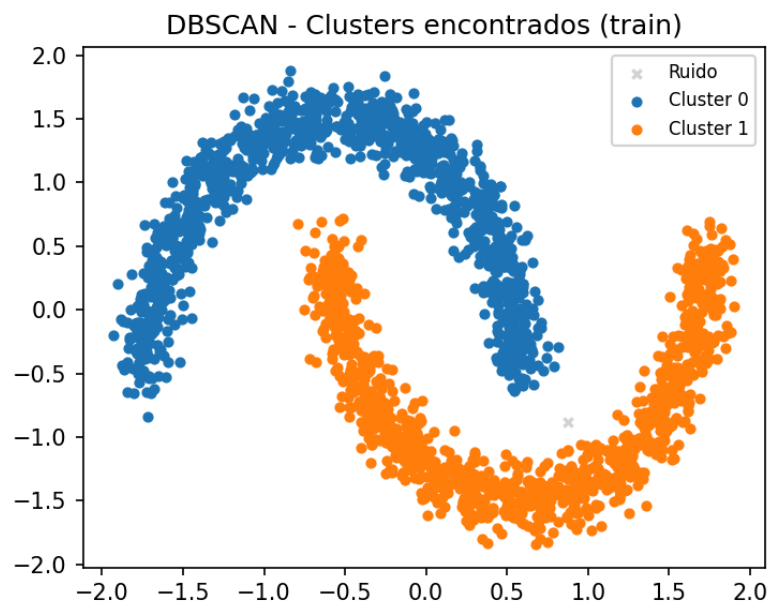


Figura 3. Clusters encontrados por DBSCAN en el conjunto de entrenamiento.

```
Silhouette sin ruido: 0.3822  
Adjusted Rand Index (vs etiquetas reales): 0.9991  
Normalized Mutual Information: 0.9974  
Homogeneity: 1.0000 | Completeness: 0.9949 | V-measure: 0.9974
```

Estas métricas comparan directamente la partición que encontró DBSCAN contra la partición real, y son las métricas estándar para evaluar clustering cuando se dispone de etiquetas.

Resultados del conjunto de prueba

Los 750 puntos de prueba se asignaron a un cluster buscando su punto núcleo de entrenamiento más cercano. Para poder comparar contra las etiquetas reales con una matriz de confusión, cada cluster se mapea a la etiqueta real más frecuente dentro de él, DBSCAN asigna valores arbitrarios a los clusters.

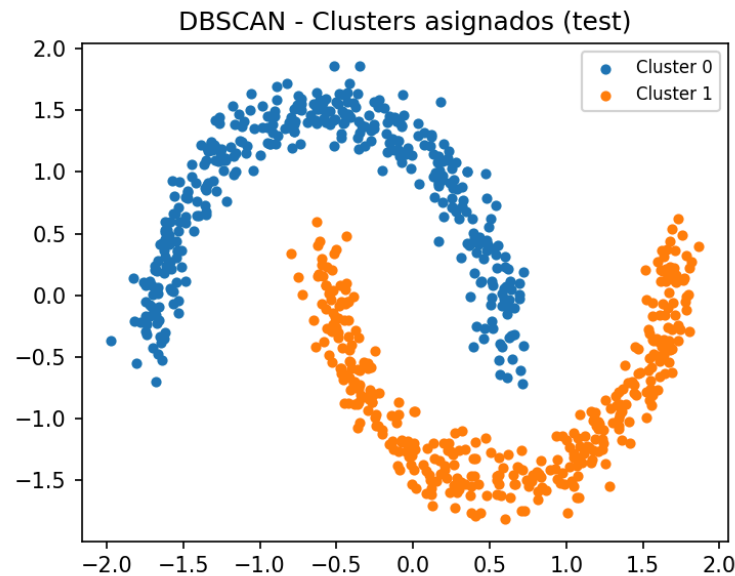


Figura 4. Clusters asignados a los puntos de prueba.

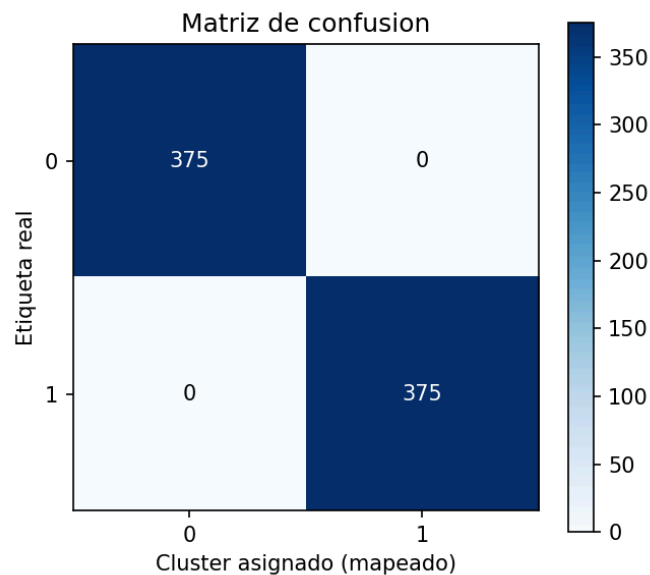


Figura 5. Matriz de confusión — conjunto de prueba (tras mapear clusters a etiquetas).

```

Classification report:
              precision    recall  f1-score   support

     0           1.000      1.000      1.000       375
     1           1.000      1.000      1.000       375

 accuracy              1.000          750
 macro avg           1.000      1.000      1.000          750
 weighted avg       1.000      1.000      1.000          750

```

Los 750 puntos de prueba se clasificaron correctamente (0 errores), y ninguno quedó marcado como ruido, todos cayeron dentro del radio ϵ de algún punto núcleo conocido.

Análisis y conclusión

El resultado (ARI = 0.999, 100% de accuracy en test) confirma que, para datos con clusters no-convexos como estas dos medias lunas, DBSCAN configurado correctamente recupera casi perfectamente la estructura real.

La elección de eps fue el factor más determinante del desempeño: el mismo min_samples con un eps mal elegido pasa de un ARI de 0.999 a uno de 0, lo que resalta por qué la técnica del k-distance plot es la forma correcta de configurar el algoritmo.

A diferencia de un clasificador supervisado, aquí no hubo entrenamiento en el sentido de ajustar pesos: DBSCAN descubre la estructura directamente a partir de la densidad de los datos, y las etiquetas reales solo se usaron para verificar qué tan bien correspondía esa estructura descubierta con la realidad.

Referencias

Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD-96 Proceedings. Scikit-learn developers. (2025). DBSCAN — scikit-learn documentation. Recuperado de <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>